

teamai-cli

把一个人的 AI 绝活，变成全团队的资产

腾讯开源 · Make Every Team AI Native · One Team. One Harness. Every Agent.

GitHub: Tencent/teamai-cli

MIT

v0.26.0 实测

16 种 AI 工具

拾遗学习流水线 · 小白指南 + 全本地闭环动手实验 (2026-10-03)

Q1 teamai-cli 是什么？一句话说清

把一个人调教好的 AI 经验 (skills / rules / hooks / MCP)，放进一个共享 Git 仓库，经 PR 评审合并后，自动同步到全团队每个人的各种 AI 工具里。

痛点场景：团队里最会用 AI 的那位老哥，他调教出的 Claude Code 技能包只躺在他自己电脑的 ~/.claude/skills/ 里。同事想学？口头转述、截图发群、抄笔记——第二天就失真了。teamai-cli 干的事就一件：让“经验”像代码一样走 Git 流水线分发。

它的 slogan 三个短语说清定位：One Team（一群人）· One Harness（同一套 AI 挽具配置）· Every Agent（Claude Code、Codex、Cursor、Copilot CLI、CodeBuddy 等 16 种工具通吃）。

Q2 没有它之前，团队怎么干？差在哪

做法	怎么分享经验	死穴
截图发群	口口相传	失真、不可执行、一周就沉底
共享文档	Wiki / 飞书文档	人得自己去翻，AI 看不见
各自复制粘贴	把 SKILL.md 拷给同事	版本分叉，改了一处其他 15 台机器不知道
teamai-cli	Git 仓库 + PR 评审 + 会话自动 pull	——（本工具要解决的）

推文 (AIGCLINK, 2026-09-09) 的点题句：teamai-cli 是**唯一把 code review 流程搬到 AI 资源上的工具**——此前 HarnessRouter、Wake、AgentsView 等解决的都是"一个人的多个 agent"，teamai 统一的是"多个人的 agent"。

Q3 它怎么做到的？核心架构：Git 仓当 registry

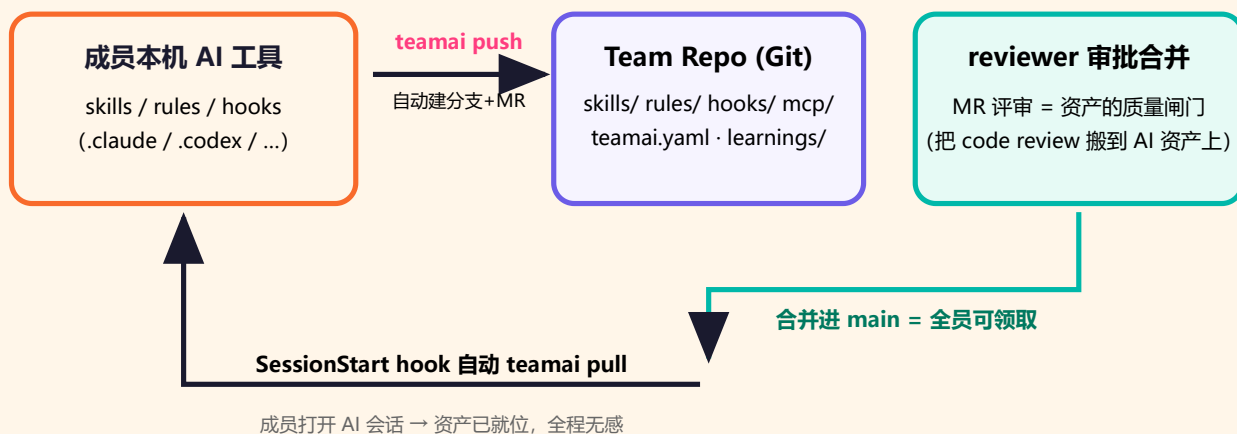


图1 · 一份资产的完整旅程：push → MR 评审 → 合并 → 自动 pull → 落入各工具原生目录

- **仓库就是普通的 Git 仓库**：GitHub / GitLab / GitCode / CNB / TGit / 私有自建全支持，不锁定任何厂商。
- **八类资产**：skills (SKILL.md 目录)、rules (团队规范 md)、docs、env (环境变量)、agents、hooks、mcp、models。
- **密钥安全**：mcp/env 配置里密钥只写 `${VAR}` 占位名，**值永远不进 Git 仓库**。

Q4 "自动同步"最后一公里怎么走？（实测 hook 注入）

本机实测：teamai init 直接往真实的 ~/.claude/settings.json 写入了四类 hook：

```
SessionStart → "...bash.exe" -lc "teamai hook-dispatch session-start --tool claude 2>/dev/null" || true
Stop          → ... hook-dispatch stop ... || true
PostToolUse   → ... hook-dispatch post-tool-use ... || true (matcher: * / Skill / TodoWrite)
UserPromptSubmit → ... hook-dispatch prompt-submit ... || true
```

注意命令结尾的 || true：同步工具挂掉绝不能连累 AI 会话——这正是知识库里 [[ClaudeCodeHooks 与 fail-open]] 说的 fail-open 模式。链路闭环：开会话 → SessionStart hook 静默执行 teamai pull → 新 skill 落进 .claude/skills/ → Claude Code 立刻可用。实验中我们眼看着本会话的技能清单凭空多出 hello-team，全程零手工操作。

Q5 16 种工具格式各异，一人一份全发？——四维分发策略

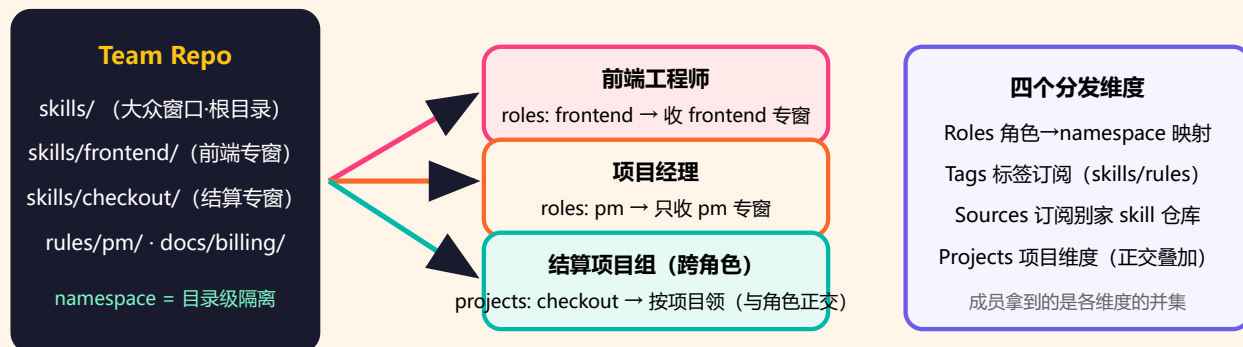


图2·namespace 分发：像食堂餐线，大众窗口 + 角色专窗 + 项目专窗

替换语义（最容易踩的理解坑）：活跃 namespace 里的同名条目会整体替换根目录条目，不做合并。

例：根目录 rules/code-style.md 是全网规范；结算项目放 rules/checkout/code-style.md 覆盖之——结算组成员拿项目版，其他人拿公司版。两个活跃 namespace 撞名时 pull 直接报错停机，宁可不更新也不静默二选一。

Q6 团队经验从哪来？——摩擦信号（全场最巧的设计）

顺畅 Session（摩擦=0）

58 次工具调用 · 一路绿灯
无打断 · 无纠正 · 无重试失败

→ 什么都不提示

较劲 Session（摩擦≠0）

你打断了 AI 两次
AI 重试失败工具 8 次 · 拒绝过 1 次调用

→ Stop hook 弹提示：/teamai share

图3 · 摩擦信号打分：只有“真正较劲过”的会话才值得沉淀

Stop hook 在会话结束时按四类信号打分：**打断 AI、纠正 AI、拒绝某次工具调用、AI 反复重试失败**。达标后终端会显示（真实文案）：“*This session may contain a problem worth documenting: you interrupted the AI twice, the AI retried failing tools 8 times.*”，并建议跑 /teamai share 把教训总结成团队 skill。每会话最多提示一次，可配置关闭。

妙处：人的干预行为本身就是知识价值的免费评分器——不用任何人主动写总结，系统只需在正确时机问一句。沉淀进 learnings/ 的知识随后可被 teamai recall 检索（子 agent 先做相关性预检，无关任务直接跳过）。

Q7 三层产品地图：分发只是第一层

层	解决什么	代表能力	状态
Team Execution	让每个 agent 按团队的方式工作	init / pull / push, 八类资产分发	稳定
Team Context	让每个 agent 理解团队	recall 检索、learnings、代码知识图谱（tree-sitter AST 轨 + 启发式轨双轨提取依赖边）	beta
Team Improvement	让每次执行都变成团队能力	摩擦信号沉淀、session 摘要、digest 周报、dashboard（干预次数 / token 用量）	beta

闭环逻辑一句话：**Execute → Understand → Learn → Self-Improve**。推文也提醒：dashboard 能看到成员干预次数与 token 用量——**引进前先想清楚这些数据的用途与边界**。

Q8 动手实验：没有 GitHub 也能跑通全闭环（全命令实测）

思路 没有真实 Git 托管端，就用「本地 bare 仓 + 自签证书 HTTPS 静态服务」模拟——git 的 dumb

HTTP 协议只需要静态文件 + `git update-server-info`。管理员端直推 main（等价于已合并的 MR），成员端走 HTTPS 只读拉取。

步骤 1 管理员端：建团队仓 + 写首批资产

目的：模拟“管理员把调教好的 skill 推上团队仓库”。

```
git init --bare team-repo.git && git clone team-repo.git seed
# 在 seed/ 写入 skills/hello-team/SKILL.md、skills/deploy-check/SKILL.md、rules/*.md
git add -A && git commit -m "feat: 首批团队资产" && git push origin main
cd ../team-repo.git && git update-server-info # dumb HTTP 需要
```

✅ 预期结果：`git ls-tree -r main --name-only` 列出 2 个 SKILL.md + 2 个 rule 文件。

步骤 2 起本地 HTTPS 服务（模拟 Git 托管端）

目的：让成员端能用 teamai 强制要求的 `https://host/group/repo.git` URL 拉取。

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 2 -nodes -subj
"//CN=127.0.0.1"
python https_server.py # 自签证书 + 静态服务，端口 8667（脚本见 exercise/）
```

✅ 预期结果：`curl -sk https://127.0.0.1:8667/team/repo.git/info/refs` 返回 main 的 commit hash。⚠️ 坑：服务根目录要取上一层目录，让 `/team/repo.git` 恰好映射到 `team-repo.git`——映射错一位 git 会报 404（看服务访问日志定位）。

步骤 3 成员端初始化（真输出）

目的：模拟新成员"一条命令加入团队"。

```
GIT_SSL_NO_VERIFY=1 TEAMAI_NONINTERACTIVE=1 \  
teamai init https://127.0.0.1:8667/team/repo.git --provider git --agent claude --  
force
```

- ✓ Using Git identity Smashwinny
 - Clone path: C:\Users\Windows\.teamai\projects\F-reminder-...\team-repo
- ✓ Team repo cloned
- ✓ Updated teamai hooks in C:\Users\Windows\.claude\settings.json
- ✓ teamai initialized successfully!

✓ 预期结果：克隆团队仓 + 注入 SessionStart/Stop/PostToolUse/UserPromptSubmit 四类 hook（都带 || true fail-open）。

步骤 4 分发资产并验证热加载（真输出）

目的：验证"资产自动落入 AI 工具原生目录"。

```
teamai pull
```

- ✓ [project] Synced 2 skills (2 new, 0 updated)
 - new: deploy-check, hello-team
- ✓ [project] Synced 2 rule(s)

✓ 预期结果：.claude/skills/ 出现两个新技能目录；正在运行的 Claude Code 会话实时看到新技能可用——零重启、零手工。

步骤 5 增量同步验证（真输出）

目的：验证“管理员改一版，全员下次会话自动拿到新版”。

```
# 管理员端把 hello-team 升到 v2 → push → 刷新 served 副本
# 成员端再拉：
teamai pull
```

- ✓ [project] Team repo: 1 file(s) changed
- ✓ [project] Synced 2 skills (all updated)

✓ 预期结果：本地 SKILL.md 变 v2，技能清单同步刷新。

步骤 6 清理还原真实环境

```
TEAMAI_NONINTERACTIVE=1 teamai uninstall --force
```

- ✓ Removed teamai hooks in C:\Users\Windows\claude\settings.json
- ✓ Removed 3 skill directories / 2 rule files / ~/.teamai/projects\...
- ✓ teamai uninstalled

✓ 预期结果：settings.json 的 hook 数组清空、注入的 skills/rules 移除——**实验工具用完即走，不留痕。**

实验踩坑实录（4 个）：① URL 必须是 https://host/group/repo.git 或 ssh 形态，本地路径 / plain HTTP 一律被拒；② dumb HTTP 只读，teamai push 的 MR 流需要真实托管端（本实验诚实记录未覆盖）；③ 项目 scope 会向上锚定最近的 git 仓根——成员目录不是独立 git 仓时，资产会装进外层仓库的 .claude/（本实验“热加载”正因此发生）；④ 服务根目录映射差一层 → git 404。

能力 这个项目给我什么？

能力	一条命令	我能拿来干什么
团队 AI 资产库	teamai init <仓库URL>	把 F:\reminder 这套学习方法 (learn-project skill) 分发给同学/同事
经验评审流	teamai push	新 skill 先过 MR 再全员生效，防"野技能"污染
无感同步	SessionStart hook 自动 pull	成员开会话即拿最新资产，零培训成本
精准分发	teamai roles / tags / source	前端不发量化 skill，按需领取
经验自动沉淀	摩擦信号 + /teamai share	踩坑会话自动提醒"要不要写进团队知识库"
团队用量看板	teamai dashboard / digest	看 AI 用量、干预热点、知识库健康

一句话总结： teamai-cli = 团队 AI 资产的 npm registry + code review 闸门 + 无感 OTA 更新三合一。Git 是 registry，MR 是闸门，hook 是最后一公里，摩擦信号是弹药库——"最会用 AI 的人"从此可以复制了。

知识入库：vault/概念/团队Harness的Git原生分发 · 摩擦信号经验沉淀 · 资源命名空间分发 | 实验材料：teamai-cli/exercise/ (RUNBOOK.md 全记录)